



An-Najah National University

Faculty of Engineering & Information Technolog

Distributed Operating Systems Project

Part 1

Students:

Hiba Burhan Banimeneh (12216976)

Shorouq Majdi Derieh (12217236)

Supervisor:

Dr.Samer Arandi

Introduction:

This project focuses on designing and implementing an online bookstore system using a microservices architecture. The system allows users to search for books, retrieve detailed book information, and purchase books through a unified frontend service. The application is implemented using Python and Flask, while SQLite is used for persistent data storage.

The system consists of three main services: the Frontend Service, the Catalog Service, and the Order Service. The Frontend Service serves as the entry point for client requests and communicates with the backend services to provide book information and process purchase operations. The Catalog Service manages book data, including titles, topics, prices, and available quantities, while the Order Service handles purchase requests and records completed orders.

To improve performance and reduce unnecessary communication with the Catalog Service, an in-memory cache is implemented within the Frontend Service. Frequently requested book information is stored in the cache, allowing subsequent requests to be served more efficiently. A cache invalidation mechanism is also implemented to ensure that outdated information is removed whenever book data changes after a purchase or update operation.

The entire application is containerized using Docker, enabling consistent deployment and execution across different environments. The project demonstrates the fundamental concepts of microservices communication, caching, data consistency, and containerized application deployment.

Technology Stack

- Operating System: Windows
- Programming Language: Python 3.x
- Database Management System: SQLite
- Web Framework: Flask

Containerization Platform: Docker

- API Testing Tool: Postman
- Version Control System: Git and GitHub

Features

- Book Catalog Management:

The system allows users to:

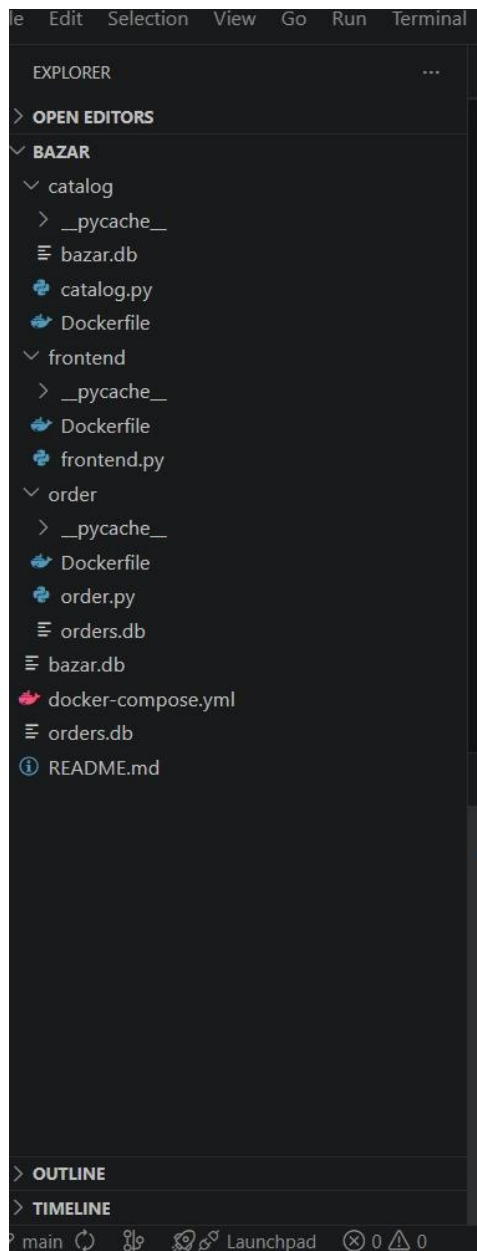
- Search books by topic.
- View detailed information about a specific book by its ID.
- Update book prices and stock based on purchasing.

- Order Management

The system also allows users to:

- Purchase books by specifying the quantity.
- Log each purchase into the orders.csv file.

Project structure



API Endpoints

Below is a list of the core API endpoints exposed by the system:

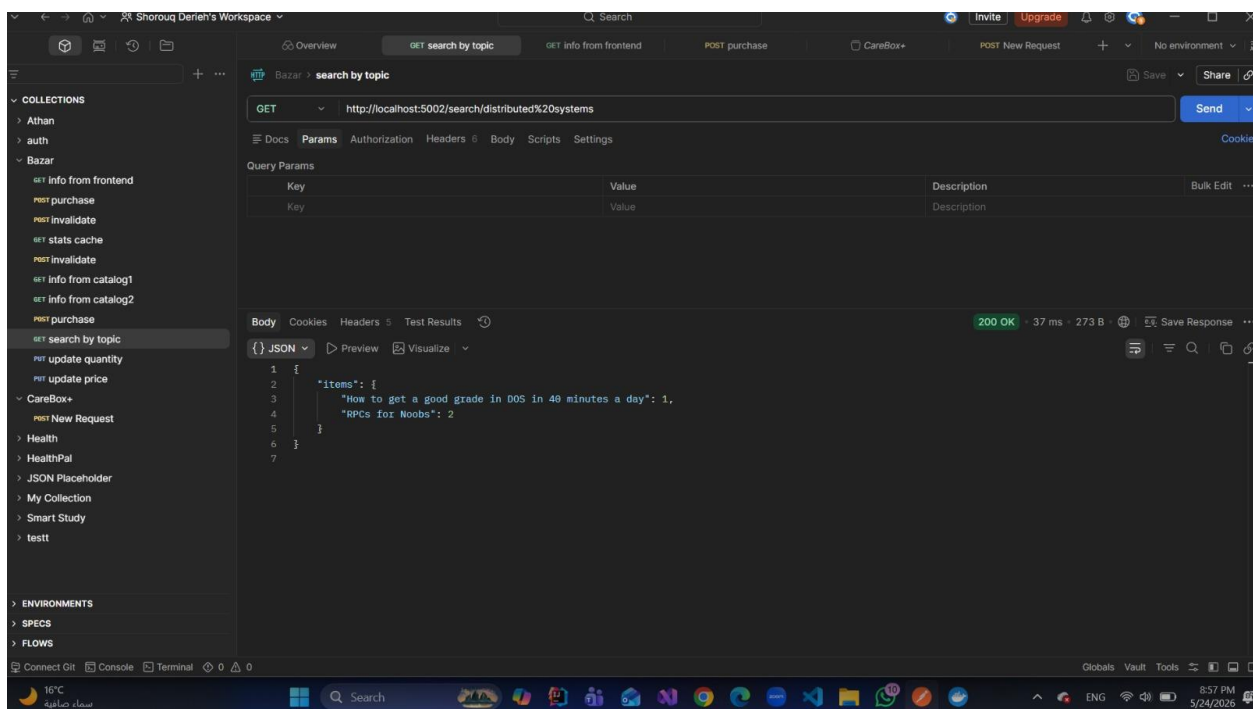
Method Endpoint		Description
GET	/search/<topic>	Searches for books that belong to the specified topic and returns matching results.
GET	/info/<item_number>	Retrieves detailed information about a specific book using its unique ID.
PUT	/update/<item_number>	Updates the price and/or stock quantity of a specific book.
POST	/purchase/<item_number>	Processes a purchase request, decreases the book stock, and records the order in the orders database.

API Testing

The following section provides step-by-step instructions on how to test the API using Postman.

- Testing the /search/:topic Endpoint
 - Method: GET
 - URL: http://localhost:5002/search/distributedsystems
 - Description: This endpoint allows searching books based on the specified topic.

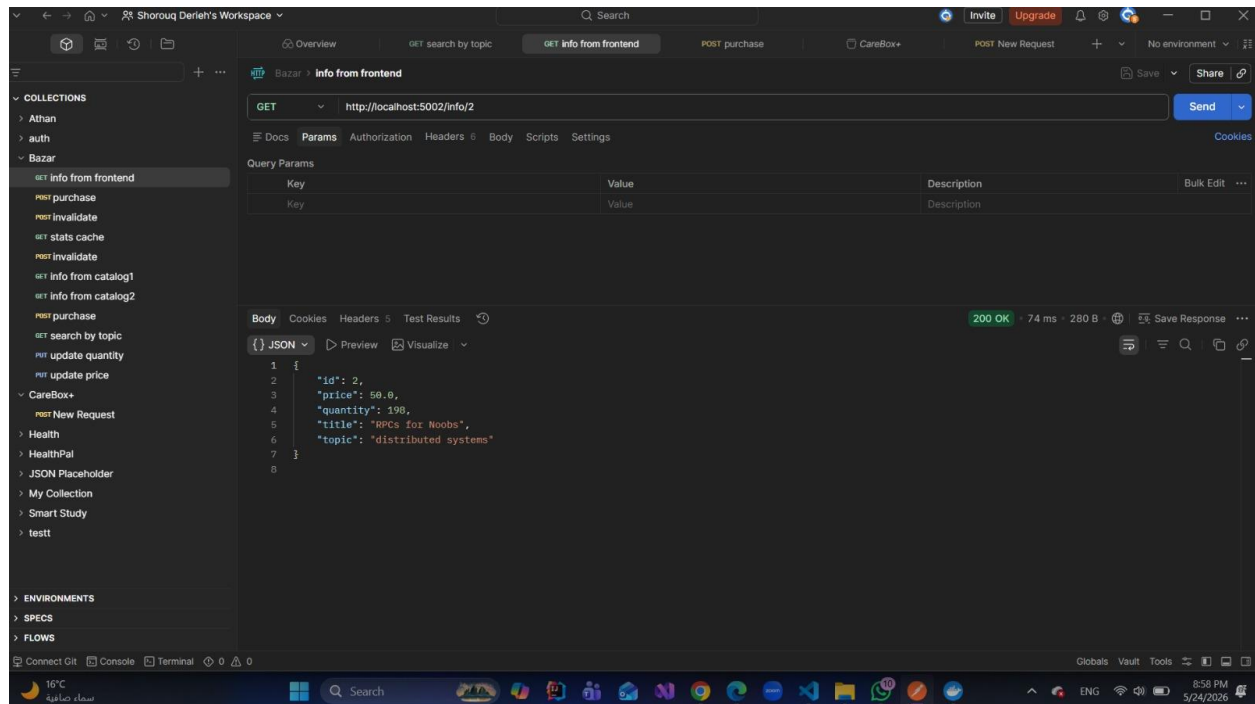
Expected Response: A list of books with the topic "distributed systems" will be returned in the response.



- Testing the /info/:item_number Endpoint
 - Method: GET

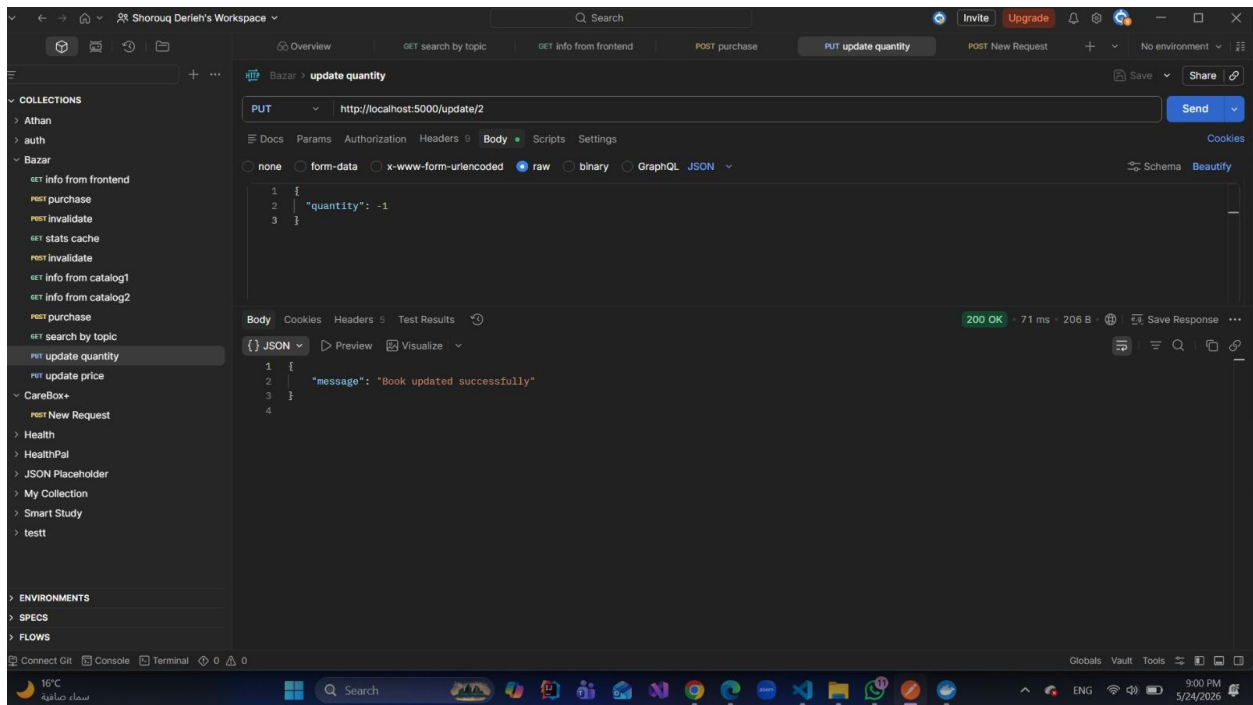
- URL: <http://localhost:5002/info/2>
- Description: This endpoint retrieves detailed information about a specific book by its ID.

Expected Response: The detailed information about the book with the specified ID will be returned in the response.



- Testing the /update Endpoint
 - Method: PUT
 - URL: <http://localhost:5002/update/2>
 - Description: This endpoint updates the price and stock of a specified book.

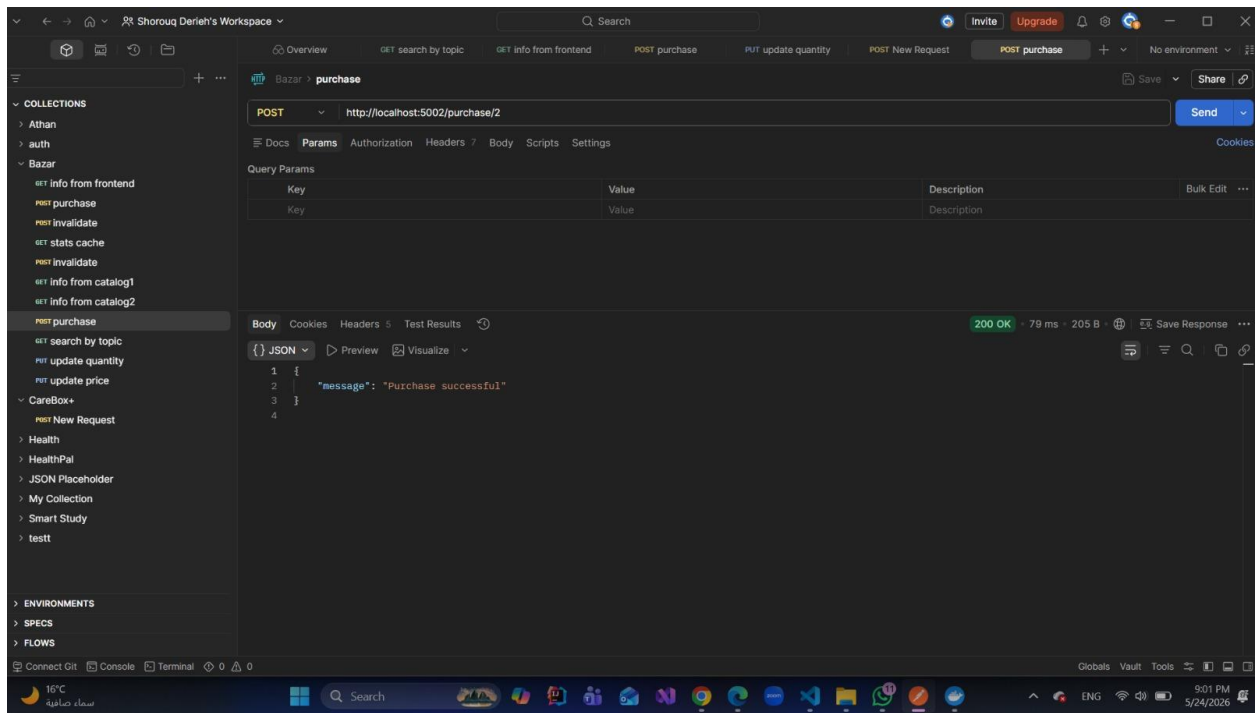
Expected Response: The updated book object, including the new price and stock values, will be returned.



```
order-1 | Order: updating Catalog (decreasing quantity)
order-1 | Order: saving order in database
order-1 | Order: purchase completed for book 2
order-1 | 172.18.0.4 - - [24/May/2026 18:01:15] "POST /purchase/2 HTTP/1.1" 200 -
```

- Testing the /purchase/:item_number Endpoint
 - Method: POST
 - URL:http://localhost:5002/purchase/2
 - Description: This endpoint processes a book purchase and decreases the stock based on the quantity specified.

Expected Response: A message indicating that the purchase was successful, along with the updated book details.



Docker Container Test

```

PS D:\Bazar> docker-compose up --build
#27 DONE 0.0s
time="2026-05-24T20:57:32+03:00" level=warning msg="Found orphan containers ([order1 order2 catalog2 catalog1]) for this project. If you removed or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up."
[+] up 6/6
✓ Image bazar-frontend      Built      27.7s
✓ Image bazar-catalog      Built      27.7s
✓ Image bazar-order        Built      27.7s
✓ Container bazar-catalog-1 Created     0.4s
✓ Container bazar-order-1   Created     0.2s
✓ Container frontend       Recreated   0.6s
Attaching to catalog-1, frontend-1, order-1
catalog-1 | * Serving Flask app 'catalog'
catalog-1 | * Debug mode: off
catalog-1 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
catalog-1 | * Running on all addresses (0.0.0.0)
catalog-1 | * Running on http://127.0.0.1:5000
catalog-1 | * Running on http://172.18.0.2:5000
catalog-1 | Press CTRL+C to quit
order-1   | * Serving Flask app 'order'
order-1   | * Debug mode: off
order-1   | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

```

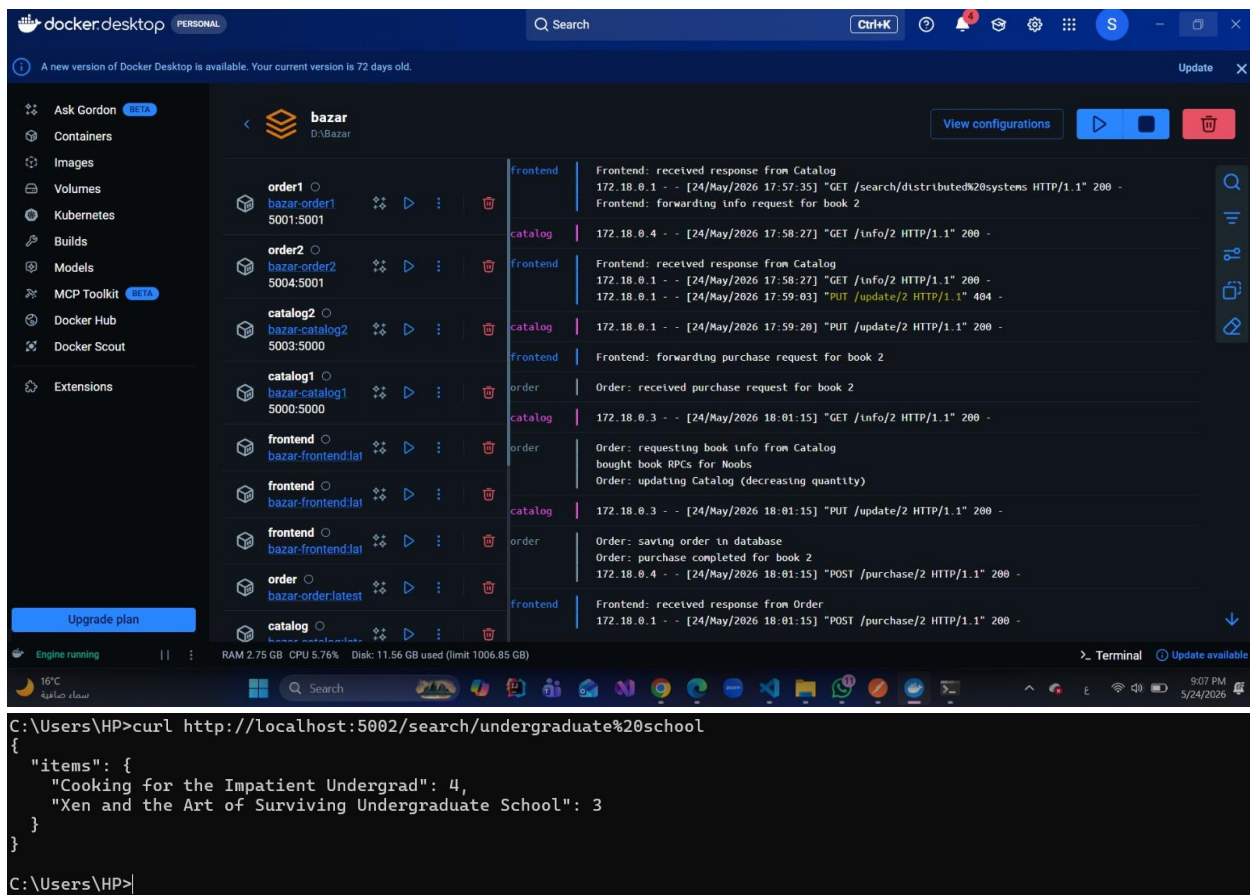
```

Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
c8761b0d3ba5   bazar-frontend "python -u frontend. ..." 8 minutes ago   Up 7 minutes   0.0.0.0:5002->5002/tcp, [::]:5002->5002/tcp
25ce9293530f   bazar-order    "python -u order.py"         8 minutes ago   Up 7 minutes   0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp
ced926eb643b   bazar-catalog  "python -u catalog.py"       8 minutes ago   Up 7 minutes   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp

C:\Users\HP>

```



File Management

The system stores its data using SQLite databases instead of CSV files. This approach provides more efficient data management, supports concurrent access, and simplifies querying and updating records.

Catalog Database (bazar.db)

The catalog database stores information about all available books in the system. Each record contains:

- Book ID
- Title
- Topic
- Price
- Available Quantity

The Catalog Service accesses this database to retrieve book information, perform searches, and update book records when necessary.

Orders Database (`orders.db`)

The orders database stores information about completed purchase transactions. Each order record contains:

- Order ID
- Book ID
- Book Title
- Purchase Date and Time

Whenever a purchase is successfully completed, the Order Service creates a new record in the orders database for tracking and auditing purposes.

Database Operations

The system performs the following database operations:

- Retrieve book information by ID.
- Search books by topic.
- Update book prices and quantities.
- Verify stock availability before purchases.
- Record completed purchase transactions.
- Maintain persistent storage across service restarts.

Error Handling

The application has basic error handling mechanisms:

- **404 Not Found:** Returned when a requested book cannot be found in the catalog database.
- **400 Bad Request:** Returned when a purchase cannot be completed due to insufficient stock or invalid request data.

- **500 Internal Server Error:** Returned when unexpected errors occur, such as database access failures or internal service errors.

Conclusion

The Book Catalog and Order Management System provides an efficient solution for managing books and processing purchases through a microservices architecture. The system supports searching books, retrieving detailed information, updating inventory, and recording orders using SQLite databases. The implementation of caching and cache invalidation improves performance while maintaining data consistency. Furthermore, Docker simplifies deployment and execution of the application. Comprehensive API testing using Postman confirmed the correctness and reliability of the implemented functionality.